



CSE 332: Computer Architecture & Organization

Assignment#3 Spring 2025

Submitted by,

Name: Muhtadina Serniabat Tasin

Student ID: 223 2160 642

Section: 5

Serial: 24

Submitted to,

Dr. Mohammad Abdul Qayum [MAQm]

Submission Date – Saturday, 15 March, 2025

Due Date – Saturday, 15 March, 2025

1. **(30 points)** Write a C program and corresponding assembly program based on MIPS ISA that reads three edges for a triangle and computes the perimeter if the input is valid. Otherwise, display that the input is invalid. The input is valid if the sum of every pair of two edges is greater than the remaining edge.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

int perimeter(int x, int y, int z); // function return perimeter
bool validity(int x, int y, int z); // function return Boolean type -
                                   // if the input is valid ...

void main(void) // display in main function
{
    int a,b,c; // variable declaration
    printf("Enter three integers: ");
    scanf("%d %d %d", &a,&b,&c); // read input
    bool valid = validity(a,b,c); // function call - validity
    if(valid)
    {
        printf("The perimeter: ");
        printf("%d", perimeter(a,b,c)); // function call - perimeter
    }
    else
        printf("The input is invalid\n");
} // end of main function

int perimeter(int x,int y,int z)
{
    return x+y+z; // return perimeter
}

bool validity(int x,int y,int z)
{
    if(x+y>z && y+z>x && z+x>y)
        return true; // return true for valid inputs

    else
        return false; // return false for invalid inputs
}
```

Assembly Program based on MIPS ISA:

```
.data
prompt:      .asciiz "Enter three integers: "
valid_msg:   .asciiz "The perimeter is: "
invalid_msg: .asciiz "The input is invalid\n"

.text
.globl main

main:
    # Print prompt
    li $v0, 4
    la $a0, prompt
    syscall

    # Read three integers
    li $v0, 5
    syscall
    move $s0, $v0 # Store first input in $s0

    li $v0, 5
    syscall
    move $s1, $v0 # Store second input in $s1

    li $v0, 5
    syscall
    move $s2, $v0 # Store third input in $s2

    # Call validity function
    move $a0, $s0
    move $a1, $s1
    move $a2, $s2
    jal validity
    move $t0, $v0 # Store result of validity check

    # If validity returned false (0), print "The input is invalid"
    beqz $t0, print_invalid

    # Otherwise, call perimeter function
    move $a0, $s0
    move $a1, $s1
    move $a2, $s2
    jal perimeter
    move $t1, $v0 # Save the perimeter result in $t1

    # Print valid message
    li $v0, 4
    la $a0, valid_msg
    syscall
```

```

    # Print the perimeter
    li $v0, 1
    move $a0, $t1 # Move the saved perimeter result to $a0
    syscall

    j exit

print_invalid:
    li $v0, 4
    la $a0, invalid_msg
    syscall

exit:
    li $v0, 10
    syscall

# Function: perimeter(int x, int y, int z)
# Returns x + y + z
perimeter:
    add $v0, $a0, $a1
    add $v0, $v0, $a2
    jr $ra

# Function: validity(int x, int y, int z)
# Checks triangle inequality rule
validity:
    add $t0, $a0, $a1 # t0 = x + y
    slt $t1, $t0, $a2 # if x + y > z ? t1 = 0
    bnez $t1, return_false

    add $t0, $a1, $a2 # t0 = y + z
    slt $t1, $t0, $a0 # if y + z > x ? t1 = 0
    bnez $t1, return_false

    add $t0, $a2, $a0 # t0 = z + x
    slt $t1, $t0, $a1 # if z + x > y ? t1 = 0
    bnez $t1, return_false

    li $v0, 1 # Return true (1)
    jr $ra

return_false:
    li $v0, 0 # Return false (0)
    jr $ra

```

Output:

```
Terminal IRQ 0 fff0000
Enter three integers: 5
12
2
The input is invalid
Enter three integers: 5
12
13
The perimeter is: 30
```

Read FIFO: 0

Integer Registers:

Registers		Registers	
Refresh		\$t1 \$9	30
pc	160	\$t2 \$10	0
\$zero \$0	0	\$t3 \$11	0
\$at \$1	0	\$t4 \$12	0
\$v0 \$2	10	\$t5 \$13	0
\$v1 \$3	0	\$t6 \$14	0
\$a0 \$4	30	\$t7 \$15	0
\$a1 \$5	12	\$s0 \$16	5
\$a2 \$6	13	\$s1 \$17	12
\$a3 \$7	0	\$s2 \$18	13
\$t0 \$8	1	\$s3 \$19	0
\$t1 \$9	30	\$s4 \$20	0
\$t2 \$10	0	\$s5 \$21	0
\$t3 \$11	0	\$s6 \$22	0
\$t4 \$12	0	\$s7 \$23	0
\$t5 \$13	0	\$t8 \$24	0
\$t6 \$14	0	\$t9 \$25	0
\$t7 \$15	0	\$k0 \$26	0
\$s0 \$16	5	\$k1 \$27	0
\$s1 \$17	12	\$gp \$28	0
\$s2 \$18	13	\$sp \$29	-2147483648
\$s3 \$19	0	\$fp \$30	0
\$s4 \$20	0	\$ra \$31	96
\$s5 \$21	0	hi	0
\$s6 \$22	0	lo	0
\$s7 \$23	0	cp0_status	536870912
\$s8 \$24	0	cp0_cause	0
\$s9 \$25	0	cp0_epc	0
\$k0 \$26	0	cp0_ebase	-2147483648
\$k1 \$27	0	cp1_fir	16192052
\$gp \$28	0	cp1_fcsr	786432

2. **(70 points)** Write an assembly program in MIPS ISA that will take input as your full name- your first and last name, then it will calculate the maximum and minimum of ASCII values (excluding the space between the first and last name) of all the letters of your name and display it in the console.

Run and test your program in CPUlator (MIPS ISA simulator: MIPS32r5 SPIM), submit a printed copy of the program, screenshot the output in the console and result in the integer registers in the Simulator. Also submit your code with .S extension as a separate file.

```
.data
prompt:      .ascii "Enter your full name (first and last name): "
buffer:      .space 100          # Buffer to store the input name
max_msg:     .ascii "Maximum ASCII value: "
min_msg:     .ascii "Minimum ASCII value: "
newline:     .ascii "\n"

.text
.globl main

main:
# Prompt for input
li $v0, 4          # syscall for print_string
la $a0, prompt     # Load address of the prompt message
syscall

# Read input (full name)
li $v0, 8          # syscall for read_string
la $a0, buffer     # Load address of the buffer
li $a1, 100        # Maximum number of characters to read
syscall

# Initialize max and min
li $t0, 0          # Initialize max to 0
li $t1, 127        # Initialize min to 127 (highest ASCII value for comparison)

# Pointer to buffer
la $t2, buffer     # Load address of the buffer into $t2

process_loop:
lb $t3, 0($t2)     # Load byte from buffer into $t3
beqz $t3, done     # If null terminator (end of string), exit loop

# Check if character is space
li $t4, ' '        # Load ASCII value of space into $t4
beq $t3, $t4, skip_space # If character is space, skip it

# Compare for max
bgt $t3, $t0, set_max # If current character > max, update max
j check_min        # Else, check for min

skip_space:
# Update min
li $t4, 127
blt $t3, $t4, set_min

set_min:
li $t1, $t3

set_max:
li $t0, $t3

check_min:
j process_loop
```

```

set_max:
move $t0, $t3           # Update max to current character
j check_min             # Jump to check for min

check_min:
blt $t3, $t1, set_min   # If current character < min, update min
j next_char             # Else, move to next character

set_min:
move $t1, $t3           # Update min to current character

next_char:
addi $t2, $t2, 1        # Move to the next character in the buffer
j process_loop          # Repeat the loop

skip_space:
addi $t2, $t2, 1        # Skip the space character
j process_loop          # Repeat the loop

done:
# Print max message
li $v0, 4               # syscall for print_string
la $a0, max_msg         # Load address of the max message
syscall

# Print max value
li $v0, 1               # syscall for print_int
move $a0, $t0           # Move max value to $a0
syscall

# Print newline
li $v0, 4               # syscall for print_string
la $a0, newline         # Load address of the newline
syscall

# Print min message
li $v0, 4               # syscall for print_string
la $a0, min_msg         # Load address of the min message
syscall

# Print min value
li $v0, 1               # syscall for print_int
move $a0, $t1           # Move min value to $a0
syscall

# Exit program
li $v0, 10              # syscall for exit
syscall

```

Output:

Terminal

IRQ 0 ffff0000

Enter your full name (first and last name): Muhtadina Tasin
Maximum ASCII value: 117
Minimum ASCII value: 10

Read FIFO: 0

Integer Registers:

Registers		Registers	
Refresh		\$t0 \$8	117
		\$t1 \$9	10
		\$t2 \$10	309
		\$t3 \$11	0
		\$t4 \$12	32
		\$t5 \$13	0
		\$t6 \$14	0
		\$t7 \$15	0
		\$s0 \$16	0
		\$s1 \$17	0
		\$s2 \$18	0
		\$s3 \$19	0
		\$s4 \$20	0
		\$s5 \$21	0
		\$s6 \$22	0
		\$s7 \$23	0
		\$t8 \$24	0
		\$t9 \$25	0
		\$k0 \$26	0
		\$k1 \$27	0
		\$gp \$28	0
		\$gp \$29	-2147483648
		\$fp \$30	0
		\$ra \$31	0
		hi	0
		lo	0
		cp0_status	536870912
		cp0_cause	0
		cp0_epc	0
		cp0_ebase	-2147483648
		cp1_fir	16192052
		cp1_fcsr	786432